

Facultad de Ingeniería
Escuela de Ingeniería Informática

Desarrollo de un software de apoyo a la práctica de programación en laboratorios docentes

Dante Augusto Chávez González

Trabajo realizado para optar al título de
Ingeniero Civil Informático

Prof. Guía: **Gabriel Astudillo Muñoz**

Valparaíso, Octubre 2026

Desarrollo de un software de apoyo a la práctica de programación en laboratorios docentes

Dante Augusto Chávez González[a,*], Gabriel Astudillo Muñoz[a]

[a]*Escuela de Ingeniería Informática, Universidad de Valparaíso, Valparaíso, Chile*

Resumen

En las asignaturas iniciales de programación del Plan Común de ingeniería, la matrícula cercana a 500 estudiantes distribuidos en múltiples secciones dificulta acompañar y medir el trabajo práctico en el laboratorio. Los docentes carecen de mecanismos para observar desde su puesto el código de los alumnos cuando surgen dudas en sala y no disponen de un registro persistente del progreso individual ni por sección. Este trabajo tiene como objetivo desarrollar un software web de apoyo a la práctica de programación en laboratorios docentes que unifique el entorno de desarrollo estudiantil con herramientas de supervisión y seguimiento académico. La solución integra JupyterHub sobre un clúster de Kubernetes de un solo nodo, instanciando contenedores efímeros y dividiendo el almacenamiento persistente en dos áreas: un espacio personal de práctica y un portafolio de evaluación (*shipping*). Esta arquitectura permite a los estudiantes programar desde un navegador web y faculta a los profesores para consultar el código en tiempo cuasi-real, publicar ejercicios con plazo límite y verificar las entregas por sección, manteniendo la capacidad de escalar hacia múltiples servidores. Finalmente, la propuesta contempla su validación mediante pruebas funcionales y de carga concurrente sobre el nodo desplegado.

Palabras clave: infraestructura híbrida, JupyterHub, orquestación de contenedores, laboratorios docentes

*Contacto: Dante Augusto Chávez González (dante.chavez@estudiantes.uv.cl)

Índice

1. Introducción **3**

2. Marco Conceptual **4**
2.1. Jupyter y JupyterHub . . . 4
2.2. Contenedores y Docker en un solo nodo . . . 5
2.3. Técnicas de escalamiento . . . 5
2.4. Orquestación con Kubernetes . . . 6
2.5. Almacenamiento persistente . . . 7
2.6. Técnicas de seguimiento del avance del estudiante . . . 7
2.6.1. Evaluación automática . . . 7
2.6.2. Control de versiones . . . 8
2.6.3. Portafolio digital . . . 8
2.6.4. Síntesis comparativa . . . 8
2.7. Estructuras de portafolio . . . 9
2.8. Síntesis . . . 10

3. Estado del Arte **10**
3.1. Codeopticon: Real-Time, One-To-Many Human Tutoring for Computer Programming . . . 11
3.2. On-Premise Kubernetes in Software Engineering Education: An Experience Report 11
3.3. JupyterHub and autograding on bare-metal lab servers . . . 11
3.4. Teaching Heterogeneous and Parallel Computing with Google Colab and Raspberry Pi Clusters . . . 12
3.5. Benefits and Pitfalls of Jupyter Notebooks in the Classroom . . . 12
3.6. Análisis Comparativo y Definición de la Brecha . . . 12

4. Definición del problema **14**
4.1. Problema . . . 14
4.2. Solución . . . 14
4.3. Objetivo general y específicos . . . 16
4.4. Importancia del trabajo . . . 16

5. Análisis de requerimientos **16**
5.1. Especificación de requisitos funcionales . . . 16
5.1.1. Módulo de Control de Acceso . . . 17
5.1.2. Módulo de Gestión Académica . . . 17
5.1.3. Módulo de Espacio de Trabajo del Estudiante . . . 17
5.1.4. Módulo de Seguimiento Docente . . . 17
5.2. Diagramas y especificación de casos de uso . . . 18
5.3. Diagramas de secuencia . . . 26
5.3.1. Autenticación y aprovisionamiento del entorno (CU-01 y CU-07) . . . 26
5.3.2. Entrega de ejercicio al portafolio de evaluación (CU-02) . . . 27
5.3.3. Consulta de código en sala y revisión de portafolios (CU-03 y CU-05) . . . 28
5.3.4. Generación de ejercicio en sala con plazo límite (CU-04) . . . 29
5.4. Diagramas de estado . . . 30

5.4.1. Ciclo de vida del contenedor del estudiante 30
5.4.2. Ciclo de vida del ejercicio y entrega al portafolio 30
5.5. Modelo conceptual de datos 31
5.6. Especificación de requisitos no funcionales 33

1. Introducción

En las asignaturas de *Introducción a la programación* y *Programación* del Plan Común de áreas de ingeniería y ciencias, que integran el plan de estudios de 10 carreras y congregan cada semestre a cerca de 500 estudiantes distribuidos en más de 10 secciones paralelas [1], acompañar el aprendizaje práctico en el laboratorio es un desafío operativo complejo. La literatura señala, entre los desafíos recurrentes de la programación introductoria, la dificultad de entregar retroalimentación oportuna durante las actividades prácticas y de acompañar individualmente a los estudiantes en cursos numerosos [2, 3]. En el laboratorio, esto se traduce en dos obstáculos concretos. En primer lugar, el docente carece de un mecanismo directo para observar desde su puesto el código que un estudiante desarrolla cuando surgen consultas, lo que retrasa la ayuda y dificulta detectar errores compartidos por la sección. En segundo lugar, los ejercicios que el alumno codifica durante la sesión no quedan registrados de manera sistemática, lo que impide un seguimiento progresivo por estudiante y por sección.

Diversos trabajos previos abordan aristas específicas de esta problemática. Por una parte, se han propuesto herramientas web para la visualización síncrona del código estudiantil en sesiones de tutoría [4], despliegues de JupyterHub con módulos de calificación sobre servidores físicos [5] y experiencias de adopción de Kubernetes en entornos universitarios [6]. Por otra parte, el análisis de entornos basados en cuadernos interactivos evidencia que delegar la instalación a los equipos personales de los alumnos introduce barreras de configuración de dependencias [7], y el uso de plataformas gratuitas en la nube pública expone a los cursos a límites de cuota y tiempos de espera [8]. No obstante, en la revisión realizada no se identificó una plataforma que articule en un mismo entorno la inspección docente del código durante el laboratorio, la recopilación continua en portafolios con plazos por sesión y la administración por secciones sobre una infraestructura de contenedores preparada para escalar.

Para abordar esta brecha, este trabajo propone el desarrollo de un software web de apoyo a la práctica de programación en laboratorios docentes. La solución despliega JupyterHub sobre un clúster de Kubernetes de un solo nodo [9], con contenedores efímeros construidos a partir de imágenes de Docker, y divide el almacenamiento persistente de cada estudiante en un *Espacio personal* de práctica diaria y un *Portafolio de evaluación* (*shipping*) [10]. Las principales contribuciones de este trabajo son:

1. **Arquitectura contenerizada en nodo único preparada para escalamiento horizontal:** diseño e implementación de una infraestructura con Kubernetes (`KubeSpawner`) y volúmenes persistentes desacoplados, que opera inicialmente en un único servidor para cubrir la demanda de las salas de laboratorio y permite escalar a múltiples servidores sin modificar los módulos del software.

2. **Mecanismo de almacenamiento dual con portafolio por sesión:** un esquema de archivos que separa la experimentación libre (*Espacio personal*) del área de entregas (*Portafolio de evaluación*), para registrar y consolidar el avance práctico clase a clase con validación de plazos y verificación ejecutable por el docente.

3. **Integración de seguimiento docente en tiempo casi real por sección académica:** módulos de gestión académica y supervisión que permiten al profesor consultar desde su puesto el código de sus estudiantes durante la clase y monitorear el cumplimiento acumulado de cada sección.

El resto del documento se organiza en cuatro secciones. La Sección 2 presenta el marco conceptual (JupyterHub, contenedores, escalamiento, Kubernetes, almacenamiento persistente y portafolios digitales). La Sección 3 revisa el estado del arte bajo el método CIR e identifica la brecha que motiva el proyecto. La Sección 4 define el problema, la solución propuesta, los objetivos y la importancia del trabajo. Finalmente, la Sección 5 especifica los requisitos y su modelamiento en UML 2.0.

2. Marco Conceptual

2.1. Jupyter y JupyterHub

Project Jupyter es una colección de software para la computación interactiva cuyos componentes permiten escribir, ejecutar y visualizar código desde un navegador web [11]. Su arquitectura distingue el entorno de ejecución individual de la gestión multiusuario.

A nivel de usuario único, el espacio de trabajo opera mediante cuatro componentes:

- **Jupyter Notebook (Cuaderno):** formato de documento estructurado en JSON que intercala celdas de código ejecutable, texto formateado (Markdown) y salidas multimedia [11].
- **JupyterLab:** interfaz gráfica web que actúa como entorno de desarrollo para editar los cuadernos y gestionar los archivos del usuario [12].
- **Kernel:** proceso independiente que ejecuta el código de las celdas (por ejemplo, el kernel IPython para Python) y devuelve los resultados a la interfaz.
- **Jupyter Server:** servidor web de usuario único que comunica la interfaz (JupyterLab) con el Kernel mediante HTTP y WebSockets [13].

Por ejemplo, en un entorno local, al iniciar una instancia de *Jupyter Server* se levanta un servicio web accesible por la dirección de bucle invertido del equipo (`http://localhost:8888`). El navegador actúa como cliente en la misma máquina y el kernel consume directamente su procesador y su memoria RAM.

Para ofrecer este espacio de trabajo a múltiples usuarios desde un servidor central se utiliza **JupyterHub**, que sirve y gestiona múltiples instancias independientes de *Jupyter Server* [14]. Su modelo multiusuario se basa en tres subsistemas:

- **Hub:** componente central que administra la base de datos y los perfiles de usuario, y controla el acceso mediante el módulo *Authenticator*, que valida credenciales contra sistemas de identidad externos.
- **Proxy HTTP:** enrutador dinámico que intercepta las peticiones web: si el usuario no está autenticado las envía al Hub, y si la sesión es válida las redirige a su servidor Jupyter privado.
- **Spawner (Aprovisionador):** componente que inicia, monitorea y detiene los servidores individuales (*Jupyter Server*). Cuando un usuario se autentica, levanta un entorno de ejecución exclusivo y aislado para esa sesión.

2.2. *Contenedores y Docker en un solo nodo*

La tecnología de contenedores empaqueta una aplicación con sus dependencias y configuración en una unidad ejecutable aislada. A diferencia de las máquinas virtuales, que emulan hardware y levantan un sistema operativo invitado completo por entorno, los contenedores se ejecutan en el espacio de usuario y comparten el núcleo (kernel) del sistema anfitrión. Esto reduce la sobrecarga de memoria y procesamiento, aísla a nivel de procesos y garantiza la reproducibilidad del entorno [15, 16].

Docker gestiona esta tecnología mediante dos entidades fundamentales:

- **Imagen:** plantilla inmutable y de solo lectura con las instrucciones y dependencias necesarias para construir un entorno de ejecución.
- **Contenedor:** instancia ejecutable de una imagen, de ciclo de vida efímero: puede iniciarse, detenerse o destruirse sin alterar la imagen base ni el sistema anfitrión [16].

Al desplegar JupyterHub con Docker en un solo nodo (un servidor físico o virtual), el *Spawner* (`DockerSpawner`) instancia un contenedor por cada usuario que inicia sesión, de modo que cada estudiante tiene un entorno de programación idéntico y aislado en la misma máquina.

Sin embargo, un nodo único tiene límites: la cantidad de contenedores simultáneos queda restringida a la CPU y la RAM de ese servidor, y no hay redundancia ante fallos de hardware [15].

2.3. *Técnicas de escalamiento*

En los sistemas distribuidos, el escalamiento es la capacidad de una arquitectura para ajustar sus recursos computacionales frente a variaciones en la carga de trabajo [17]. Existen dos técnicas principales para escalar software:

- **Escalamiento vertical (Scale-up):** aumentar la capacidad de un único nodo, añadiendo hardware (CPU, RAM o almacenamiento) o asignando una mayor cuota de estos recursos a un proceso. Tiene un límite físico, el de la máquina anfitriona, y mantiene un único punto de fallo [18].
- **Escalamiento horizontal (Scale-out):** aumentar la capacidad agregando instancias de software o nuevos nodos (servidores) que trabajan de forma coordinada. Permite superar los límites de una sola máquina y distribuir el riesgo de fallos [18, 17].

Se distingue además entre **escalabilidad**, propiedad del sistema para soportar mayor carga agregando recursos de forma proporcional, y **elasticidad**, capacidad de aprovisionar y liberar esos recursos de forma dinámica y automática según la demanda [17].

En un entorno de contenedores con Docker en **un solo nodo**, el escalamiento puede aplicarse a nivel local con *Docker Engine* y *Docker Compose*. El escalamiento vertical se logra modificando los límites de recursos del contenedor (parámetros como `--memory` o `--cpus`), con los que Docker ajusta los grupos de control (*cgroups*) del kernel anfitrión [19]. El horizontal, dentro de un único nodo, se logra con la especificación de despliegue (`deploy`) de *Docker Compose*, cuyo atributo `replicas` instancia varios contenedores idénticos de un mismo servicio [20].

Aun así, el escalamiento horizontal sigue acotado a los recursos de esa máquina. Superar ese límite exige escalar a nivel de infraestructura (múltiples nodos), lo que requiere un sistema de orquestación de clústeres [15].

2.4. Orquestación con Kubernetes

La orquestación de contenedores es la gestión automatizada de su ubicación, ciclo de vida, comunicación en red y disponibilidad sobre un conjunto de máquinas interconectadas (clúster) [15].

Una primera opción de orquestador, integrada en el motor de Docker, es **Docker Swarm** (*Swarm mode*), que agrupa varias instancias de Docker Engine en nodos administradores (*manager nodes*), que mantienen el estado del enjambre, y nodos trabajadores (*worker nodes*), que solo ejecutan los contenedores asignados [21].

Kubernetes es una plataforma de código abierto para la orquestación de cargas de trabajo contenerizadas que abstrae los recursos físicos de múltiples servidores bajo una API unificada [22]. Un clúster de Kubernetes se compone de dos elementos principales:

- **Plano de Control (Control Plane):** servicios que gestionan el estado global del clúster: el servidor de la API (`kube-apiserver`), el almacén clave-valor (`etcd`), el gestor de controladores (`kube-controller-manager`), que mantiene el estado deseado de los objetos, y el planificador (`kube-scheduler`), que evalúa los recursos de cada servidor y decide dónde ubicar cada carga de trabajo [22].
- **Nodos (Nodes):** máquinas físicas o virtuales donde se despliegan las aplicaciones. Cada nodo ejecuta un agente (`kubelet`), que se comunica con el plano de control, y un motor de contenedores compatible con el estándar OCI (como *containerd*), que ejecuta imágenes construidas con herramientas como Docker [22].

En Kubernetes, la unidad mínima de despliegue no es el contenedor individual, sino el **Pod**, que encapsula uno o más contenedores que comparten almacenamiento y dirección IP dentro del clúster [22].

Para automatizar el escalamiento y la elasticidad, Kubernetes incorpora tres mecanismos generales [23]:

1. **Horizontal Pod Autoscaler (HPA):** escala horizontalmente servicios sin estado, aumentando o disminuyendo el número de réplicas de un mismo Pod según el consumo de CPU o memoria.
2. **Vertical Pod Autoscaler (VPA):** escala verticalmente, ajustando las solicitudes y límites de CPU y memoria de los contenedores de un Pod.
3. **Cluster Autoscaler:** escala horizontalmente a nivel de infraestructura, incorporando o liberando nodos de trabajo cuando el `kube-scheduler` detecta Pods pendientes por falta de recursos.

JupyterHub se integra con Kubernetes mediante **KubeSpawner** (parte de *Zero to JupyterHub with Kubernetes*) [9]. A diferencia de un microservicio que replica un mismo Pod con HPA, cada usuario de JupyterHub necesita un entorno con estado propio, por lo que `KubeSpawner` crea un Pod individual por cada estudiante autenticado. En un despliegue inicial de un solo nodo, la elasticidad consiste en crear el Pod al iniciar sesión y destruirlo por inactividad (*culling*), liberando memoria. Para escalar a varios servidores se agregan nodos al clúster y el `kube-scheduler` distribuye los Pods entre las máquinas disponibles, sin alterar el software.

2.5. Almacenamiento persistente

El sistema de archivos interno de un contenedor es efímero por diseño. Al instanciarlo desde una imagen, el motor añade una capa superior de lectura y escritura (*writable layer*) para los archivos creados o modificados durante la sesión. Esa capa está acoplada al contenedor: si este se elimina, o si el Pod es reubicado en otro nodo del clúster, sus datos se destruyen de forma permanente [24, 25].

Para conservar los archivos más allá del ciclo de vida del contenedor existen abstracciones de almacenamiento persistente:

- **Persistencia en Docker (un solo nodo):** mediante *bind mounts*, que montan un directorio del sistema anfitrión dentro del contenedor, o **Volúmenes de Docker**, directorios desacoplados del ciclo de vida del contenedor y administrados por el motor en un área dedicada del disco anfitrión [24].
- **Persistencia en Kubernetes (múltiples nodos):** como en un clúster el Pod puede crearse en cualquier servidor, el almacenamiento no puede depender del disco local de un nodo. Kubernetes separa la provisión del consumo con dos objetos: el **PersistentVolume (PV)**, recurso de almacenamiento en red (como NFS o Ceph) aprovisionado en el clúster, y el **PersistentVolumeClaim (PVC)**, solicitud de almacenamiento de un usuario [10]. Al iniciar sesión con `KubeSpawner`, el PVC personal del estudiante se vincula a su Pod y monta sus archivos sin importar en qué nodo se ejecute [26].

Para este software, el almacenamiento persistente se estructura en dos áreas, una para el trabajo del estudiante y otra para la supervisión docente:

1. **Espacio personal de desarrollo:** directorio privado de lectura y escritura del estudiante para su trabajo en curso: crear scripts libremente, ejecutar pruebas de forma iterativa y modificar cuadernos interactivos durante las sesiones de laboratorio.
2. **Espacio de entrega y revisión (shipping):** área que resguarda las versiones finalizadas o seleccionadas para revisión formal. Separarla del espacio de experimentación evita que el trabajo a evaluar se altere accidentalmente mientras el estudiante sigue practicando, y ofrece al cuerpo docente un acceso estructurado y centralizado.

2.6. Técnicas de seguimiento del avance del estudiante

Este apartado identifica técnicas para verificar que los estudiantes realizan el trabajo práctico asignado y conocer, clase a clase, los resultados que obtienen. Se describen tres, resumidas en la Tabla 1, y al final se comparan según criterios derivados de los problemas de este trabajo para determinar cuál es la más adecuada.

2.6.1. Evaluación automática

Consiste en revisar automáticamente el código que entregan los estudiantes. Paiva et al. [27] revisan el estado del arte de esta técnica: tipos de ejercicios soportados, medidas de seguridad, técnicas de prueba, retroalimentación generada e información que el sistema entrega al docente para comprender y optimizar el aprendizaje. Identifican una nueva etapa apoyada en técnicas de análisis estático y en la contenerización.

2.6.2. *Control de versiones*

Un sistema de control de versiones como Git permite al docente observar la evolución del trabajo del estudiante. Tu et al. [28], con la experiencia de 8 docentes de la Universidad de Auckland en 15 cursos, señalan que el docente con acceso puede clonar el repositorio de cada estudiante y revisar su historial completo de commits, lo que permite evaluar más allá del código final. También advierten que, con estudiantes novatos en Git, las pruebas fallidas del autocorrector de GitHub Classroom aparecen con marcas rojas que pueden preocuparlos, pues podrían interpretarlas como un commit fallido.

2.6.3. *Portafolio digital*

La tercera técnica es el portafolio digital, una colección de trabajos del estudiante acompañada de reflexión sobre lo aprendido. Olstad et al. [29] lo estudian como evaluación formativa en cursos de computación en Noruega, a partir de las reflexiones finales de 67 estudiantes, una encuesta nacional y observaciones de su desarrollo. Sus resultados indican que ayuda a los estudiantes a reconocer las competencias adquiridas y que varios reportaron mayor motivación para aprender. Garcia [30] analiza portafolios digitales en un curso de diseño y desarrollo web con 176 estudiantes de pregrado, que completaron tareas semanales de programación culminadas en un portafolio programado por ellos mismos. Parte de la observación de que los estudiantes suelen dedicar poco esfuerzo a estos portafolios por considerar poco desafiante su componente de escritura, y reporta que la experiencia previa con portafolios digitales influye positivamente en el compromiso, la motivación y la apropiación del aprendizaje.

Tabla 1: Técnicas de seguimiento del avance del estudiante y fuentes que las sustentan.

Técnica	**Qué registra o evalúa**	**Fuente**
Evaluación automática	Verificación automatizada del código entregado y retroalimentación generada por el sistema	[27]
Control de versiones	Historial de commits de cada estudiante, accesible al docente	[28]
Portafolio digital	Colección de trabajos del estudiante con reflexión sobre lo aprendido	[29], [30]

2.6.4. *Síntesis comparativa*

La comparación se centra en el registro persistente y el seguimiento por estudiante y por sección, pues la inspección del código en tiempo real es un mecanismo síncrono de sala y no de recopilación histórica. Como se busca registrar el trabajo práctico y sus resultados clase a clase, se definen cinco criterios:

- **Evidencia del trabajo por sesión:** comprobar que el estudiante desarrolló las actividades de cada clase, más allá de validar si la solución final es correcta.
- **Evaluación de resultados del código:** verificar directamente la correctitud funcional o las salidas del programa del estudiante en cada sesión.
- **Registro histórico longitudinal:** conservación estructurada y acumulativa de las entregas durante todo el periodo académico, para su revisión periódica.

- **Seguimiento por estudiante y por sección:** organizar y consultar el avance de forma individual y por sección.
- **Facilidad para estudiantes novatos:** bajo esfuerzo técnico adicional para registrar el avance, relevante porque la asignatura reúne a alumnos de primer año de distintas carreras [1].

La Tabla 2 compara las tres técnicas según estos criterios. Las valoraciones corresponden a un análisis propio a partir de la literatura revisada.

Tabla 2: Comparación de las técnicas de seguimiento según los criterios definidos.

Técnica	**Evidencia por sesión**	**Resultados del código**	**Registro longitudinal**	**Estudiante y sección**	**Facilidad para novatos**
Evaluación automática	Media	Alta	Media	Media	Alta
Control de versiones	Alta	Media	Alta	Media	Baja
Portafolio digital	Alta	Media	Alta	Alta	Media

Ninguna técnica destaca en todos los criterios. La evaluación automática es la que mejor informa los resultados de cada clase y la más sencilla para novatos, pero se orienta a calificar la correctitud del producto entregado más que a recopilar el trabajo práctico realizado. El control de versiones conserva un registro histórico detallado, aunque exige aprender la herramienta, y Tu et al. [28] advierten que los novatos en Git pueden malinterpretar las señales de pruebas fallidas, una carga adicional en una asignatura introductoria. El portafolio digital reúne el trabajo de cada clase ordenado por estudiante y por sección en un registro longitudinal, lo que lo hace la técnica más alineada con el problema planteado. Su principal riesgo es el señalado por Garcia [30]: el poco esfuerzo que suelen dedicar los estudiantes a la redacción. Este riesgo se reduce si el portafolio se construye con los ejercicios de código que el estudiante ya realiza en clase, sin exigirle escritura adicional.

Por estas razones, se selecciona el portafolio digital para dar seguimiento al trabajo de los estudiantes, alojado en el espacio de entrega y revisión (*shipping*) de la subsección 2.5. Se concibe como la recopilación de los ejercicios de cada clase, que el docente revisa clase a clase o de forma periódica. Se espera que su carácter obligatorio incentive a completar los ejercicios durante la sesión y permita considerar el porcentaje de entregas al final del semestre. Estas decisiones son política docente del curso: el software las habilita mediante el registro y seguimiento de las entregas, pero no las produce por sí mismo.

Los estudios de portafolios revisados evalúan principalmente la percepción y el compromiso de los estudiantes, por lo que esta elección se sustenta en la adecuación a los criterios definidos y no en evidencia previa de que un portafolio obligatorio aumente por sí solo el trabajo en sala.

2.7. Estructuras de portafolio

Seleccionado el portafolio digital, falta definir cómo se organizan y seleccionan sus evidencias. La literatura sobre evaluación con portafolios en educación superior y ciencias de la computación distingue tres estructuras según su propósito y los artefactos que almacenan [29, 30]:

- **Portafolio de vitrina o resultados (*Showcase portfolio*):** reúne solo los mejores trabajos o productos finales al término de un periodo, para demostrar el máximo nivel de competencia alcanzado, sin borradores, errores previos ni ejercicios incompletos [29].

- **Portafolio de proceso o trabajo (*Process / Working portfolio*):** estructura cronológica que almacena de forma continua las actividades prácticas, ejercicios en progreso e iteraciones sesión a sesión, para documentar la evolución del aprendizaje y permitir al docente observar el trabajo completado y las dificultades recurrentes [29].

- **Portafolio de evaluación (*Assessment portfolio*):** estructura estandarizada que organiza los artefactos según criterios, rúbricas o unidades temáticas predefinidas por la institución o el cuerpo docente, para verificar sistemáticamente el cumplimiento de los requisitos del curso [30].

Dado que Garcia [30] advierte el bajo compromiso de los estudiantes frente a componentes extensos de redacción reflexiva, se define como **decisión de diseño propia** una estructura híbrida entre el **portafolio de proceso** y el **portafolio de evaluación**, construida solo con los artefactos de código (cuadernos interactivos de Jupyter) de cada sesión de laboratorio, sin informes escritos adicionales.

Este diseño vincula los dos espacios definidos en la subsección 2.5: el estudiante resuelve los ejercicios en su *Espacio personal* y, antes del vencimiento del plazo de la clase, envía la versión correspondiente al espacio de entrega y revisión (*shipping*), llamado en el sistema *Portafolio de evaluación*. Allí los archivos se organizan en una jerarquía de directorios indexada por sección académica, estudiante y número de sesión, se consolida una copia inmutable tras el cierre del plazo y el docente puede inspeccionarlos mediante copias de prueba ejecutables.

2.8. Síntesis

JupyterHub provee el espacio de trabajo multiusuario [14]. Docker y Kubernetes aíslan cada entorno y permiten iniciar con un nodo para escalar después [15, 9], mientras que los volúmenes persistentes habilitan el almacenamiento dual [10] sobre el que se construye el portafolio digital, técnica seleccionada por su mejor ajuste a los criterios definidos (Tabla 2) [29, 30]. Estos elementos conforman el marco con el que se evalúan los trabajos previos en la siguiente sección.

3. Estado del Arte

El desafío central de este proyecto es la dificultad para medir la práctica de programación de los estudiantes en las asignaturas de programación del Plan Común, que se traduce en dos problemas (detallados en la Sección 4):

1. Dificultad para acceder al código del estudiante en tiempo real, lo que impide resolver sus consultas de forma oportuna.

2. Falta de persistencia y trazabilidad de la práctica: no existe un registro sistemático del progreso por estudiante ni por sección.

Las soluciones y tecnologías actuales se analizan con el método CIR (Contexto, Implementación, Resultados), que estructura la revisión de cada trabajo en tres ejes que aseguran una evaluación objetiva y comparable:

- Contexto: problema que los autores buscaban resolver y escenario en que se aplicó la investigación.

- Implementación: herramientas, arquitectura y metodología usadas para construir o desplegar la solución.
- Resultados: evidencia empírica y métricas concretas (rendimiento, tiempos, capacidad, encuestas) obtenidas al aplicar el sistema, que permiten validar su efectividad.

Así, la revisión se estandariza y de cada documento se extraen solo los datos relevantes para contrastarlos con los problemas planteados.

3.1. Codeopticon: Real-Time, One-To-Many Human Tutoring for Computer Programming

Guo (2015) [4] aborda directamente el primer problema: la dificultad de acceder al código del estudiante y dar asistencia personalizada en cursos de programación a gran escala. Identifica que los tutores no pueden observar directamente la depuración de cada alumno para ayudar a tiempo.

Implementó una aplicación web con HTML5 y WebSockets que amplía la plataforma Online Python Tutor. A diferencia de infraestructuras contenerizadas complejas, presenta al profesor un panel centralizado con visualizaciones en tiempo real del editor de cada estudiante, que destaca errores de compilación y muestra los cambios del código mediante diferencias de texto (*diffs*). Incluye además un historial interactivo para revisar las acciones previas del alumno antes de iniciar un chat de asistencia integrado.

En una sesión de 30 minutos, un profesor llegó a monitorear a 226 estudiantes distintos en total, con 2 a 3 conversaciones de ayuda simultáneas en promedio. Cualitativamente, todos los participantes consideraron el sistema una alternativa auténtica a la tutoría presencial en laboratorio, con ventajas como medir el progreso colectivo de la clase y verificar si todos avanzaban de forma constante.

3.2. On-Premise Kubernetes in Software Engineering Education: An Experience Report

Aunque se enmarca en la enseñanza de ingeniería de software y el despliegue de aplicaciones, el reporte de Linhuber et al. (2026) [6] aporta antecedentes directos sobre la viabilidad de operar clústeres de Kubernetes propios (*on-premise*) en una universidad, evitando la dependencia y los costos variables de la nube pública.

Los autores desplegaron un clúster local sobre infraestructura universitaria e integraron el control de acceso con el sistema central de identidad de la institución, de modo que los estudiantes se autenticaban con sus credenciales universitarias y sus recursos se aislaban mediante permisos por rol.

Mantener una plataforma contenerizada local redujo la sobrecarga operativa del equipo técnico frente a la provisión tradicional de máquinas virtuales individuales. Aunque el trabajo no aborda portafolios ni supervisión de código en cursos introductorios, sus resultados respaldan la factibilidad técnica de una infraestructura institucional basada en Kubernetes e integrada con la autenticación universitaria.

3.3. JupyterHub and autograding on bare-metal lab servers

Flemming (2022) [5] aborda el segundo problema desde la gestión de tareas, buscando automatizar la evaluación y registrar el trabajo de los estudiantes. Analiza los desafíos técnicos de implementar JupyterHub junto con la herramienta de autocalificación nbgrader para un entorno con múltiples cursos y profesores simultáneos.

A diferencia de un enfoque contenerizado, la plataforma se desplegó directamente sobre el sistema operativo de un servidor físico (*bare-metal*), con entornos virtuales de Python y un servidor LDAP para autenticar usuarios. Para que la calificación soportara múltiples clases, los administradores debieron modificar el código fuente de la herramienta y configurar servicios de tokens en la API.

Sin embargo, el despliegue sobre la capa base del sistema operativo generó severos conflictos técnicos, en particular con el módulo de autenticación de Linux (PAM). La gestión de sesiones paralelas provocaba errores de permisos denegados y fallos de segmentación (*segfaults*), impedía aislar correctamente los recursos y obligó a deshabilitar el manejo nativo de sesiones. Esto refuerza la necesidad de usar contenedores aislados.

3.4. Teaching Heterogeneous and Parallel Computing with Google Colab and Raspberry Pi Clusters

Para evaluar la alternativa de delegar la ejecución a servicios externos sin administrar servidores propios, se revisa la experiencia de Xu (2023) [8]. Aunque se desarrolla en un curso de computación paralela y heterogénea, expone las implicancias operativas de usar cuadernos Jupyter en nubes públicas gratuitas durante sesiones docentes.

En la primera etapa de la asignatura, los estudiantes usaron Google Colab para escribir y ejecutar código desde el navegador, guardando sus cuadernos en cuentas personales de Google Drive, sin servidores locales en la facultad.

Aunque esta modalidad eliminó los costos iniciales de equipamiento institucional, varios estudiantes señalaron en las encuestas que los límites de uso del servicio gratuito podían provocar largas esperas y reducir su productividad, especialmente al trabajar con cuadernos grandes que demandan recursos como GPU. Extrapolado al contexto de este trabajo, depender de cuentas personales en nubes comerciales deja los archivos del alumno fuera del control académico de la universidad y sin integración con secciones ni seguimiento docente. El límite de cuota, en cambio, probablemente pese menos en ejercicios introductorios, que demandan pocos recursos.

3.5. Benefits and Pitfalls of Jupyter Notebooks in the Classroom

Finalmente, sobre el entorno que usará el estudiante, Johnson (2020) [7] aborda los obstáculos técnicos y pedagógicos que surgen cuando los alumnos instalan y gestionan Jupyter Notebooks en sus propios equipos. Destaca que el formato de programación literaria de la herramienta es ideal para el aprendizaje iterativo y facilita desarrollar habilidades de comunicación técnica.

No obstante, las instalaciones locales imponen altas barreras de entrada por la compleja gestión de dependencias y la incomprensión del modelo cliente-servidor. Además, evaluar código mediante el intercambio de archivos locales introduce riesgos de seguridad por la ejecución accidental de código malicioso. Para mitigarlos, el autor recomienda como mejor práctica configurar servidores remotos que estandaricen el entorno de trabajo, lo que fundamenta ofrecer un espacio en la nube gestionado por la institución.

3.6. Análisis Comparativo y Definición de la Brecha

De la revisión realizada se derivan cinco criterios que un sistema debe cumplir para acompañar y registrar la práctica de programación en el Plan Común de ingeniería:

- **Monitoreo en Sala (Tiempo casi real):** observar desde el puesto del docente el código que el estudiante desarrolla durante la sesión, con intervalos breves de actualización y sin envío manual de archivos.

- **Trazabilidad y Persistencia:** registro sistemático del progreso en un portafolio longitudinal y almacenamiento persistente del espacio de trabajo durante el semestre.
- **Aislamiento de Entornos:** espacios de ejecución encapsulados que eviten conflictos de dependencias o permisos entre usuarios concurrentes.
- **Escalabilidad:** capacidad de soportar más sesiones simultáneas y expandir los recursos computacionales.
- **Gestión Institucional:** integrar autenticación universitaria unificada y estructurar a los usuarios en una jerarquía académica de secciones, docentes y estudiantes.

La Tabla 3 contrasta los enfoques de la literatura revisada con el diseño propuesto.

Tabla 3: Análisis comparativo de soluciones para entornos de programación educativos.

Enfoque / Solución	**Monitoreo en Sala**	**Trazabilidad**	**Aislamiento**	**Escalabilidad**	**Gestión Institucional**
Plataformas Web Síncronas [4]	Alta (Tiempo real)	Media (Solo sesión)	Baja	Baja	Nula (No autenticado)
K8s On-Premise Educativo [6]	Nula (Sin entorno)	Baja (Sin portafolio)	Alta (Pods)	Alta (Clúster)	Alta (SSO y roles)
JupyterHub + nbgrader Bare-Metal [5]	Baja (Solo entregas)	Alta (Tareas)	Baja (Conflictos OS)	Baja (Servidor único)	Media (Adaptación manual)
Nubes Públicas (Ej. Colab) [8]	Nula	Media (Cuenta personal)	Alta	Alta (Sujeta a cuotas)	Nula (Cuentas externas)
Entornos Locales (BYOD) [7]	Nula	Baja	Alta (Físico)	Nula (Equipo propio)	Nula
Diseño Propuesto	**Media (retardo $\leq$ 60 s)**	**Alta (Portafolio por sesión)**	**Alta (Contenedores)**	**Media (Escalable por diseño, 1 nodo)**	**Alta (Módulo de Secciones)**

Los enfoques revisados resuelven por separado aristas específicas del problema. Herramientas como Codeopticon [4] destacan en la visibilidad síncrona del editor, pero operan sobre fragmentos de código acotados, sin almacenamiento persistente entre sesiones, estructura de secciones ni autenticación universitaria.

Delegar la ejecución a los equipos personales introduce barreras de configuración de dependencias y riesgos al intercambiar archivos ejecutables [7]. Los servicios gratuitos en la nube pública eliminan la instalación local [8], pero sus restricciones de uso pueden afectar la continuidad de las sesiones y las cuentas personales desvían la custodia de los archivos fuera de la institución.

En las plataformas institucionales basadas en JupyterHub, extensiones como `nbgrader` permiten distribuir tareas y recolectar entregas para calificarlas [5]. Sin embargo, su despliegue *bare-metal* presenta conflictos de aislamiento entre sesiones concurrentes y requiere modificaciones manuales para operar con múltiples cursos y profesores [5]. Además, `nbgrader` está concebido para evaluar tareas formales y no permite al docente observar el código en desarrollo en el espacio personal del alumno durante la clase. Por su parte, los clústeres de Kubernetes locales confirman la robustez del aislamiento por contenedores y la integración con credenciales institucionales [6], pero requieren una capa de software académica que abstraiga la gestión de contenedores para docentes y estudiantes de primer año.

Limitaciones identificadas y oportunidad de mejora: en la revisión realizada no se identificó una solución que combine en una misma plataforma el monitoreo docente del código durante la sesión, el registro continuo en portafolios con plazos por clase y el agrupamiento por secciones sobre contenedores orquestados. Existe, por tanto, la oportunidad de desarrollar un software apoyado en JupyterHub y Kubernetes que automatice el aprovisionamiento de entornos aislados y articule los módulos de control de acceso institucional, gestión de secciones, espacio de trabajo dual y seguimiento docente.

4. Definición del problema

4.1. Problema

En el Plan Común de áreas de ingeniería y ciencias se imparten las asignaturas de *Introducción a la programación* y *Programación*, cuyo propósito es que los estudiantes adquieran competencias iniciales de programación. Por ser una actividad eminentemente práctica, su diseño curricular combina bloques de teoría y sesiones de práctica en laboratorio [2]. En el laboratorio, la práctica se materializa principalmente en el análisis y ejecución de códigos de ejemplo y la resolución de ejercicios en sala [31]. Medirla es necesario por dos razones. Primero, permite al docente ajustar el contenido y el ritmo de la clase según el avance real de los alumnos [3]. Segundo, como los cursos reúnen a estudiantes de diversas disciplinas, la ejercitación supervisada es el mecanismo para que generalicen los conceptos teóricos mediante la práctica [2, 31].

Operativamente, estas asignaturas se caracterizan por su escala masiva y su metodología presencial en laboratorio. Cada semestre abarcan 10 carreras de ingeniería [1] y una matrícula cercana a los 500 estudiantes, distribuidos en más de 10 secciones paralelas a cargo de más de 10 profesores. Como los estudiantes desarrollan los ejercicios directamente durante el bloque de clases, esta escala plantea como desafío principal la dificultad para medir y acompañar su práctica de programación.

De este desafío se desprenden dos problemas concretos:

1. **Dificultades para acceder al código del estudiante durante la clase:** cuando surgen dudas, el docente carece de un mecanismo expedito para ver desde su puesto el código que el estudiante escribe en ese instante, lo que retrasa la resolución de consultas y la detección de errores comunes en la sala.

2. **Ausencia de registro de la práctica:** los ejercicios desarrollados en la sesión no se almacenan de manera sistemática, por lo que no existe un seguimiento histórico del trabajo ni por estudiante ni por sección.

En conjunto, no poder inspeccionar el código oportunamente en sala ni contar con seguimiento continuo por estudiante y sección dificulta la dinámica que debe tener el proceso de enseñanza-aprendizaje en estas asignaturas [3]. Además, como evidenció el estado del arte (Sección 3), las plataformas existentes abordan el aprovisionamiento de infraestructura o la visualización en vivo de forma aislada, sin integrarlos bajo una estructura administrativa de múltiples secciones.

4.2. Solución

Se propone un software de apoyo a la práctica de programación en el que el estudiante trabaja en un entorno accesible desde el navegador y el docente puede observar y medir esa práctica. Las decisiones de arquitectura responden a los hallazgos del estado del arte. Primero, se adopta un entorno centralizado en un servidor institucional para eliminar las barreras de configuración local reportadas por Johnson [7] y evitar los límites de cuota y las cuentas externas de las nubes públicas evidenciados por Xu [8]. Segundo, para evitar los fallos de aislamiento y los conflictos de permisos a nivel de sistema operativo que Flemming documenta en despliegues *bare-metal* [5], los entornos de usuario se encapsulan en contenedores efímeros. Tercero, siguiendo la factibilidad demostrada por Linhuber et al. para infraestructuras universitarias *on-premise* [6], el alcance contempla un único servidor con un clúster de Kubernetes de un solo nodo integrado con JupyterHub (`KubeSpawner`) e imágenes construidas con Docker, y deja la arquitectura preparada

para escalar agregando nuevos nodos sin modificar el software. Finalmente, para cubrir la necesidad de supervisión en el aula expuesta por Guo [4] dentro de un entorno completo de Jupyter, se incorpora el acceso docente en tiempo casi real junto con el registro por sesión en portafolios.

La Figura 1 presenta el diagrama de alto nivel de la solución: sus tres actores (Estudiantes, Profesores y Administradores) y la interacción entre sus cuatro módulos.

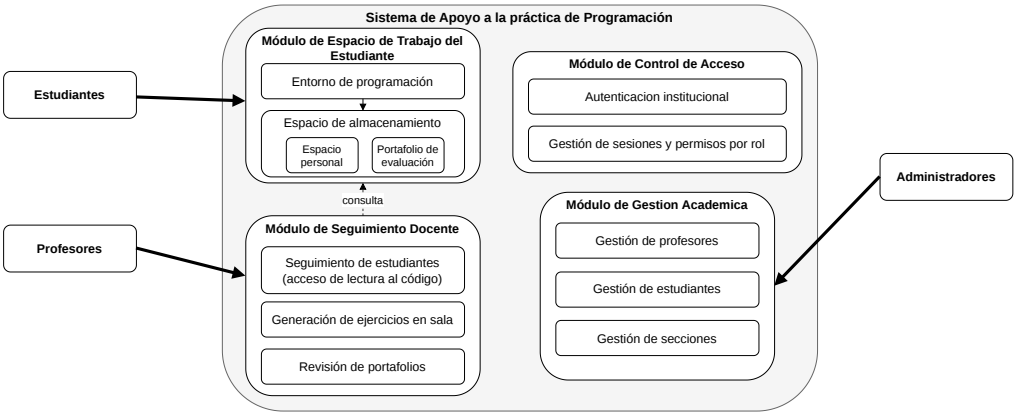


Figura 1: Diagrama de alto nivel de la solución propuesta para el Sistema de Apoyo a la Práctica de Programación.

El software se organiza en los siguientes cuatro módulos:

- **Módulo de Espacio de Trabajo del Estudiante:** provee al alumno el *Entorno de programación* interactivo en el navegador, conectado a su *Espacio de almacenamiento* persistente. Según el marco conceptual, este se divide en un **Espacio personal**, para el desarrollo libre y la experimentación diaria, y un **Portafolio de evaluación** (área de entrega o *shipping*), donde se almacenan los ejercicios de cada sesión.

- **Módulo de Seguimiento Docente:** se vincula en modo de consulta con el almacenamiento de los estudiantes y ofrece tres funcionalidades al profesor: el *Seguimiento de estudiantes*, con acceso de lectura en tiempo casi real al código durante la sesión para resolver consultas, la *Generación de ejercicios en sala* con plazo límite, para distribuir actividades a la sección, y la *Revisión de portafolios*, para verificar y ejecutar en copias de prueba el avance acumulado por estudiante, sección y sesión.

- **Módulo de Control de Acceso:** gestiona el ingreso y la operación segura mediante la *Autenticación institucional* (cuentas de Microsoft de la universidad) y la *Gestión de sesiones y permisos por rol*, que restringe las vistas y recursos de estudiantes, profesores y administradores.

- **Módulo de Gestión Académica:** operado por los administradores, estructura la carga académica del semestre mediante la *Gestión de profesores*, la *Gestión de estudiantes* y la *Gestión de secciones*, que establecen el agrupamiento lógico entre cada docente y sus alumnos.

4.3. Objetivo general y específicos

Objetivo general

Desarrollar un software web de apoyo a la práctica de programación en laboratorios docentes, soportado sobre una infraestructura de contenedores, que permita a los profesores monitorear el código de los estudiantes durante la clase y realizar el seguimiento de su avance por estudiante y por sección mediante portafolios digitales.

Objetivos específicos

1. Diseñar la arquitectura del sistema, integrando el aprovisionamiento de contenedores con JupyterHub sobre Kubernetes, el almacenamiento persistente en dos espacios (personal y de entrega) y el modelo de datos por secciones académicas.

2. Implementar los módulos de acceso institucional, gestión académica, espacio de trabajo del estudiante y seguimiento docente, que permitan monitorear el código en sala y revisar portafolios.

3. Validar el sistema mediante pruebas funcionales y de carga en un nodo, con 40 a 80 contenedores concurrentes, y estimar su escalabilidad frente a una matrícula de 500 usuarios.

4.4. Importancia del trabajo

La importancia de este trabajo se fundamenta desde un **punto de vista social**, al incidir en las condiciones de enseñanza-aprendizaje de cerca de 500 estudiantes de primer año y más de 10 docentes de las 10 carreras que cursan el Plan Común de áreas de ingeniería y ciencias [1].

Primero, ante la dificultad para observar el código en clase, el sistema busca mejorar la interacción entre docentes y estudiantes en las salas de laboratorio. Con alumnos de distinta familiaridad previa, las dudas de sintaxis o lógica suelen detener el trabajo hasta que el profesor llega al puesto del estudiante. Se espera que la consulta remota del código en tiempo casi real agilice la orientación y facilite identificar errores frecuentes en el grupo.

Segundo, ante la ausencia de un registro continuo de la práctica, los portafolios por estudiante y por sección buscan evitar que las dificultades de aprendizaje pasen inadvertidas en grupos masivos. La evidencia sesión a sesión permite al cuerpo docente detectar tempranamente a los estudiantes que no completan las actividades, antes de las evaluaciones formales, y ajustar el ritmo y el apoyo pedagógico de cada sección.

5. Análisis de requerimientos

Esta sección especifica los requerimientos funcionales y no funcionales del sistema y su modelamiento en UML 2.0, a partir de los tres actores de la solución (Estudiante, Profesor y Administrador) y sus cuatro módulos funcionales.

5.1. Especificación de requisitos funcionales

Los requisitos funcionales (RF) definen las capacidades y servicios que el software debe proveer a sus usuarios. A continuación se detallan los 13 RF, agrupados por módulo.

5.1.1. *Módulo de Control de Acceso*

- **RF-01 (Autenticación institucional):** el sistema debe permitir a estudiantes, profesores y administradores iniciar sesión con su correo y contraseña institucionales mediante el proveedor de identidad de Microsoft de la universidad.
- **RF-02 (Gestión de sesiones y control de permisos por rol):** el sistema debe mantener activa la sesión del usuario autenticado, identificar su rol (Estudiante, Profesor o Administrador) y restringir el acceso a las vistas, contenedores y recursos autorizados para ese perfil.

5.1.2. *Módulo de Gestión Académica*

- **RF-03 (Gestión CRUD de profesores):** el sistema debe permitir al administrador realizar operaciones CRUD (crear, leer, actualizar y eliminar o deshabilitar) sobre los registros de profesores.
- **RF-04 (Gestión CRUD de estudiantes):** el sistema debe permitir al administrador realizar operaciones CRUD (crear, leer, actualizar y eliminar o deshabilitar) sobre los registros de estudiantes, incluida su importación mediante nóminas académicas.
- **RF-05 (Gestión CRUD de secciones académicas):** el sistema debe permitir al administrador realizar operaciones CRUD sobre las secciones académicas de cada semestre, asociando a cada sección uno o más profesores y una lista de estudiantes matriculados.

5.1.3. *Módulo de Espacio de Trabajo del Estudiante*

- **RF-06 (Aprovisionamiento del entorno de programación):** el sistema debe instanciar y asignar un contenedor individual con el entorno interactivo de Jupyter cuando el estudiante accede a su espacio de trabajo desde el navegador.
- **RF-07 (Gestión de archivos en el espacio personal):** el sistema debe proveer al estudiante un directorio persistente de lectura y escritura (*Espacio personal*) donde pueda crear, modificar, ejecutar y eliminar cuadernos interactivos y scripts durante sus sesiones de práctica.
- **RF-08 (Obtención de ejercicios en sala):** el sistema debe permitir al estudiante visualizar los ejercicios publicados por el docente de su sección y descargar sus archivos base a su *Espacio personal* para resolverlos.
- **RF-09 (Entrega y actualización de ejercicios en el portafolio con cierre por plazo):** el sistema debe permitir al estudiante enviar un ejercicio resuelto desde su *Espacio personal* a su *Portafolio de evaluación* (área de *shipping*). Mientras no venza el plazo límite fijado por el profesor, el estudiante podrá reentregar y actualizar la versión almacenada. Vencido el plazo, el sistema debe bloquear nuevos envíos y consolidar la última versión recibida como inmutable.

5.1.4. *Módulo de Seguimiento Docente*

- **RF-10 (Visualización de secciones y estudiantes asignados):** el sistema debe presentar al profesor el listado de secciones bajo su cargo junto con la nómina de estudiantes matriculados en cada una.

- **RF-11 (Acceso de lectura en tiempo casi real al código del estudiante en sala):** el sistema debe permitir al profesor acceder en solo lectura a los archivos del *Espacio personal* de cualquier estudiante de sus secciones. Con un intervalo de autoguardado de hasta 60 segundos en el entorno del alumno, el docente observará el progreso del código en tiempo casi real para resolver consultas durante el laboratorio.
- **RF-12 (Generación de ejercicios en sala con tiempo límite):** el sistema debe permitir al profesor crear y publicar ejercicios prácticos para una sección específica, definiendo su archivo base o instrucciones y configurando una fecha y hora límite para su entrega (por ejemplo, al finalizar el bloque de clases o al término del día).
- **RF-13 (Revisión y ejecución de portafolios de evaluación):** el sistema debe permitir al profesor consultar los trabajos del *Portafolio de evaluación* filtrados por sección, estudiante y sesión de laboratorio, y abrir los archivos bajo el esquema de inspección ejecutable de prueba sin alterar el respaldo original de la entrega.

La Tabla 4 resume los RF con su identificador, nombre, módulo y actor principal.

Tabla 4: Resumen de los requisitos funcionales del sistema.

Código	**Nombre del requisito**	**Módulo asociado**	**Actor**
RF-01	Autenticación institucional	Control de Acceso	Todos
RF-02	Gestión de sesiones y permisos por rol	Control de Acceso	Todos
RF-03	Gestión CRUD de profesores	Gestión Académica	Administrador
RF-04	Gestión CRUD de estudiantes	Gestión Académica	Administrador
RF-05	Gestión CRUD de secciones académicas	Gestión Académica	Administrador
RF-06	Aprovisionamiento del entorno	Espacio de Trabajo	Estudiante
RF-07	Gestión de archivos en espacio personal	Espacio de Trabajo	Estudiante
RF-08	Obtención de ejercicios en sala	Espacio de Trabajo	Estudiante
RF-09	Entrega y actualización de ejercicios en el portafolio con cierre por plazo	Espacio de Trabajo	Estudiante
RF-10	Visualización de secciones y estudiantes	Seguimiento Docente	Profesor
RF-11	Acceso de lectura en tiempo casi real al código del estudiante en sala	Seguimiento Docente	Profesor
RF-12	Generación de ejercicios con tiempo límite	Seguimiento Docente	Profesor
RF-13	Revisión de portafolios de evaluación	Seguimiento Docente	Profesor

5.2. Diagramas y especificación de casos de uso

El modelamiento de casos de uso en UML 2.0 representa las interacciones entre los actores externos y el sistema para alcanzar objetivos funcionales observables. Los tres actores son:

- **Estudiante:** usuario matriculado en una sección que ejercita la programación en su contenedor personal y entrega sus actividades al portafolio de evaluación.

- **Profesor:** docente a cargo de una o más secciones que publica ejercicios en sala, observa el código de sus alumnos en tiempo casi real y revisa los portafolios de evaluación.
- **Administrador:** encargado técnico o coordinador académico que gestiona el registro de profesores, estudiantes y la conformación de las secciones por semestre.

La Figura 2 presenta el diagrama general de casos de uso. Con el enfoque de objetivos de usuario de UML 2.0, los 13 RF se agrupan en 7 casos de uso (CU-01 a CU-07) que representan flujos de interacción completos. El acceso y la interacción inicial del alumno en su contenedor (RF-06, RF-07 y RF-08) forman CU-01, la visualización de nóminas por sección (RF-10) es el paso inicial de los casos de uso docentes (CU-03, CU-04 y CU-05), las operaciones CRUD (RF-03, RF-04 y RF-05) se consolidan en CU-06, y la seguridad y los permisos (RF-01 y RF-02) conforman el caso transversal de inclusión CU-07. Los demás RF corresponden uno a uno a CU-02 (RF-09), CU-03 (RF-11), CU-04 (RF-12) y CU-05 (RF-13).

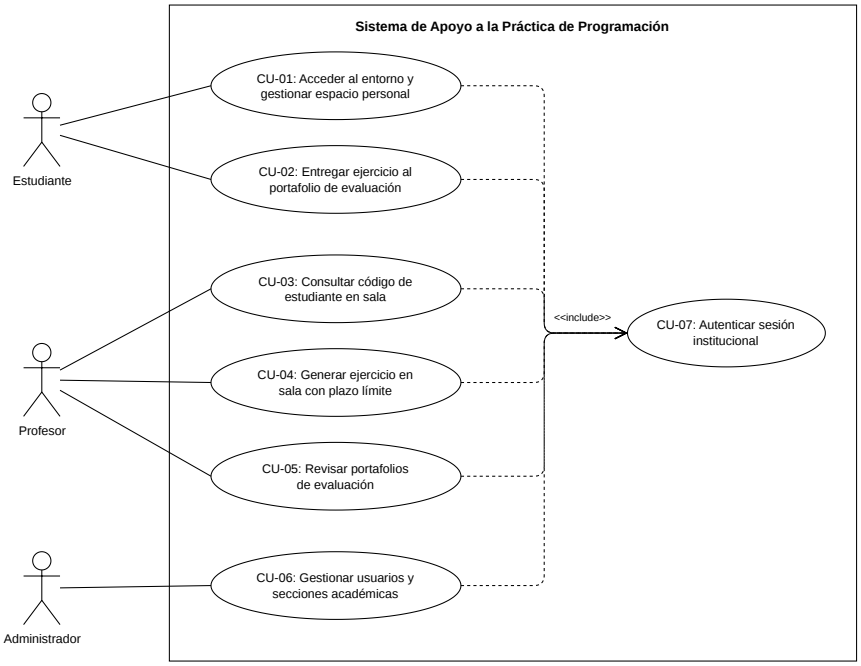


Figura 2: Diagrama de casos de uso del Sistema de Apoyo a la Práctica de Programación.

Las Tablas 5 a 11 especifican cada caso de uso (actores, propósito, tipo, descripción, referencias cruzadas con los RF y curso normal de eventos).

Tabla 5: Especificación del caso de uso CU-01.

Caso de Uso	**CU-01: Acceder al entorno y gestionar espacio personal**
Actores	Estudiante (iniciador).
Propósito	Proveer al estudiante un entorno de programación en contenedor para desarrollar y ejecutar ejercicios prácticos en su directorio privado.
Tipo	Primario, esencial.
Descripción	El estudiante ingresa a la plataforma, el sistema entrega un contenedor individual con Jupyter y monta su volumen persistente. El estudiante descarga los ejercicios de su sección y edita o ejecuta su código dentro de su Espacio personal.
Referencias cruzadas	RF-06, RF-07, RF-08. Incluye a CU-07.
Curso normal de eventos	
Acción del Actor	**Respuesta del Sistema**
1. El Estudiante solicita acceder a su espacio de trabajo desde el navegador web.	2. El sistema ejecuta el caso de uso CU-07 para verificar la sesión y el rol de Estudiante.
	3. El sistema solicita a JupyterHub (mediante KubeSpawner) instanciar el contenedor del estudiante y montar su volumen persistente. 4. El sistema despliega la interfaz de programación mostrando el directorio del Espacio personal y los ejercicios disponibles para su sección.
5. El Estudiante selecciona un ejercicio publicado en espacio virtual para trabajar en él.	6. El sistema copia los archivos base del ejercicio hacia el Espacio personal del estudiante.
7. El Estudiante edita las celdas de código, ejecuta el programa y guarda los cambios.	8. El kernel del contenedor procesa las instrucciones, retorna las salidas en pantalla y almacena los archivos en el Espacio personal.

Tabla 6: Especificación del caso de uso CU-02.

Caso de Uso	**CU-02: Entregar ejercicio al portafolio de evaluación**
Actores	Estudiante (iniciador).
Propósito	Transferir o actualizar un ejercicio desde el Espacio personal hacia el Portafolio de evaluación dentro del plazo límite de la sesión.
Tipo	Primario, esencial.
Descripción	El estudiante selecciona un archivo de su Espacio personal para enviarlo a la actividad de la clase. Si el plazo sigue vigente, el sistema almacena la copia en el área de portafolio (*shipping*), permitiendo reemplazarla mediante nuevas entregas hasta la hora de cierre, momento en que queda consolidada e inmutable.
Referencias cruzadas	RF-09. Incluye a CU-07.
Curso normal de eventos	
Acción del Actor	**Respuesta del Sistema**
1. El Estudiante selecciona la opción de entregar un ejercicio desde su entorno de trabajo.	2. El sistema valida la sesión activa mediante CU-07 y consulta los ejercicios con plazo abierto para la sección del estudiante.
3. El Estudiante indica el archivo de su Espacio personal que corresponde a la actividad y confirma el envío.	4. El sistema verifica que la fecha y hora actual del servidor sean anteriores o iguales a la fecha y hora límite configuradas por el profesor.
	5. El sistema copia el archivo hacia el directorio del Portafolio de evaluación indexado por sección, estudiante y sesión (actualizando la entrega previa si existía una dentro del plazo) e impide la edición manual directa sobre dicho directorio. 6. El sistema registra la nueva marca de tiempo de la entrega y muestra un mensaje de confirmación al estudiante.

Tabla 7: Especificación del caso de uso CU-03.

Caso de Uso	**CU-03: Consultar código de estudiante en sala**
Actores	Profesor (iniciador).
Propósito	Visualizar en modo de solo lectura y en tiempo casi real el código que un estudiante está desarrollando en su Espacio personal durante la sesión de laboratorio.
Tipo	Primario, esencial.
Descripción	Durante la clase, el profesor selecciona una de sus secciones y elige a un estudiante de la nómina para inspeccionar los archivos de su Espacio personal (actualizados mediante autoguardado periódico de hasta 60 segundos) y resolver dudas sobre su código.
Referencias cruzadas	RF-10, RF-11. Incluye a CU-07.
Curso normal de eventos	
Acción del Actor	**Respuesta del Sistema**
1. El Profesor accede al módulo de seguimiento docente.	2. El sistema verifica las credenciales y el rol de Profesor mediante CU-07 y despliega las secciones asignadas al docente.
3. El Profesor selecciona la sección en curso y elige a un estudiante del listado.	4. El sistema consulta en modo de solo lectura el directorio del Espacio personal del estudiante seleccionado.
5. El Profesor selecciona el cuaderno o script de código en el que está trabajando el alumno.	6. El sistema renderiza la última versión autoguardada del archivo en la pantalla del profesor sin bloquear el entorno de edición del estudiante.

Tabla 8: Especificación del caso de uso CU-04.

Caso de Uso	**CU-04: Generar ejercicio en sala con plazo límite**
Actores	Profesor (iniciador).
Propósito	Crear y distribuir una actividad práctica para una sección académica definiendo su fecha y hora límite de recepción.
Tipo	Primario, esencial.
Descripción	El profesor publica un nuevo ejercicio para una de sus secciones, adjunta el cuaderno o archivo base de código y fija el tiempo límite hasta el cual se permitirán entregas en el portafolio.
Referencias cruzadas	RF-10, RF-12. Incluye a CU-07.
Curso normal de eventos	
Acción del Actor	**Respuesta del Sistema**
1. El Profesor selecciona una de sus secciones y elige la opción de crear un ejercicio en sala.	2. El sistema valida la sesión mediante CU-07 y despliega el formulario de configuración de la actividad.
3. El Profesor ingresa el título, adjunta el archivo base del ejercicio y define la fecha y hora límite de entrega.	4. El sistema valida que los campos estén completos y que la fecha límite sea posterior a la hora actual del servidor.
5. El Profesor confirma la publicación del ejercicio.	6. El sistema almacena el ejercicio asociado a la sección, crea el directorio de recepción en los portafolios y habilita la descarga para los estudiantes del grupo.

Tabla 9: Especificación del caso de uso CU-05.

Caso de Uso	**CU-05: Revisar portafolios de evaluación**
Actores	Profesor (iniciador).
Propósito	Consultar y ejecutar el registro de entregas de los estudiantes de una sección para monitorear su avance y verificar el funcionamiento de su código.
Tipo	Primario, esencial.
Descripción	El profesor accede al área de revisión de portafolios, filtra por sección, estudiante o sesión de laboratorio, y abre el código entregado en un entorno que permite su lectura, edición de prueba y ejecución para comprobar que la salida del programa sea correcta.
Referencias cruzadas	RF-10, RF-13. Incluye a CU-07.
Curso normal de eventos	
Acción del Actor	**Respuesta del Sistema**
1. El Profesor ingresa a la vista de revisión de portafolios y selecciona una sección asignada.	2. El sistema verifica el acceso mediante CU-07 y recupera el registro de entregas del área de portafolio (*shipping*) para esa sección.
	3. El sistema presenta una matriz con la nómina de estudiantes, los ejercicios realizados en el semestre y el estado de entrega de cada uno.
4. El Profesor selecciona la entrega específica de un estudiante para una sesión determinada.	5. El sistema carga una instancia del archivo almacenado en el Portafolio de evaluación junto con su fecha y hora de envío, habilitando permisos de lectura, escritura y ejecución para que el docente compruebe si la salida del código coincide con el resultado esperado.

Tabla 10: Especificación del caso de uso CU-06.

Caso de Uso	**CU-06: Gestionar usuarios y secciones académicas**
Actores	Administrador (iniciador).
Propósito	Mantener actualizado el registro de profesores, estudiantes y secciones académicas mediante operaciones CRUD.
Tipo	Primario, esencial.
Descripción	El administrador crea o actualiza las secciones del semestre, registra cuentas de profesores y estudiantes (de forma individual o por nómina) y asigna los participantes a cada sección.
Referencias cruzadas	RF-03, RF-04, RF-05. Incluye a CU-07.
Curso normal de eventos	
Acción del Actor	**Respuesta del Sistema**
1. El Administrador accede al módulo de gestión académica.	2. El sistema verifica el rol de Administrador mediante CU-07 y muestra el panel de administración de profesores, estudiantes y secciones.
3. El Administrador ejecuta una operación CRUD sobre el registro de profesores o estudiantes (o carga una nómina de alumnos).	4. El sistema valida el formato de los correos institucionales y actualiza los registros en la base de datos.
5. El Administrador crea o selecciona una sección académica y le asocia uno o más profesores junto con su lista de estudiantes.	6. El sistema registra los vínculos de la sección y configura los permisos de consulta docente y los directorios de almacenamiento correspondientes.

Tabla 11: Especificación del caso de uso CU-07.

Caso de Uso	**CU-07: Autenticar sesión institucional**
Actores	Estudiante, Profesor, Administrador.
Propósito	Validar la identidad del usuario a través de su cuenta institucional de Microsoft y autorizar los permisos correspondientes a su rol.
Tipo	Secundario, de inclusión.
Descripción	El sistema verifica si el usuario posee una sesión activa. En caso contrario, delega la validación del correo y contraseña al proveedor de identidad de Microsoft de la universidad y carga los permisos del perfil.
Referencias cruzadas	RF-01, RF-02. Incluido por CU-01, CU-02, CU-03, CU-04, CU-05 y CU-06.
Curso normal de eventos	
Acción del Actor	**Respuesta del Sistema**
1. El Actor intenta ejecutar una funcionalidad protegida dentro del sistema.	2. El sistema comprueba si existe un token de sesión activo. Si no existe, redirige al navegador hacia la pasarela de inicio de sesión de Microsoft.
3. El Actor ingresa su correo electrónico institucional y su contraseña.	4. El proveedor de Microsoft valida las credenciales y retorna la confirmación de identidad al sistema.
	5. El sistema consulta el rol asociado al correo institucional en su base de datos (Estudiante, Profesor o Administrador) y habilita las vistas autorizadas.

5.3. Diagramas de secuencia

Las interacciones temporales entre los actores y los componentes internos se especifican en cuatro diagramas de secuencia, centrados en los contenedores, el almacenamiento persistente dual y el seguimiento docente: autenticación y aprovisionamiento del entorno (CU-01 y CU-07), entrega de ejercicios con validación de plazo (CU-02), consulta de código en sala y revisión ejecutable de portafolios (CU-03 y CU-05) y generación de ejercicios en sala (CU-04). CU-06 corresponde a transacciones CRUD estándar entre la interfaz de administración y la base de datos académica, por lo que no se diagrama.

5.3.1. Autenticación y aprovisionamiento del entorno (CU-01 y CU-07)

La Figura 3 muestra el inicio de sesión institucional y la creación del contenedor del estudiante. Al solicitar acceso, el Módulo de Control de Acceso (integrado en el Hub) redirige la petición al proveedor de identidad de Microsoft de la universidad. Validadas las credenciales, el Hub verifica el rol e instruye a `KubeSpawner` instanciar el contenedor, y antes de habilitar la interfaz de Jupyter se monta el volumen persistente del alumno con su *Espacio personal* y su *Portafolio de evaluación*.

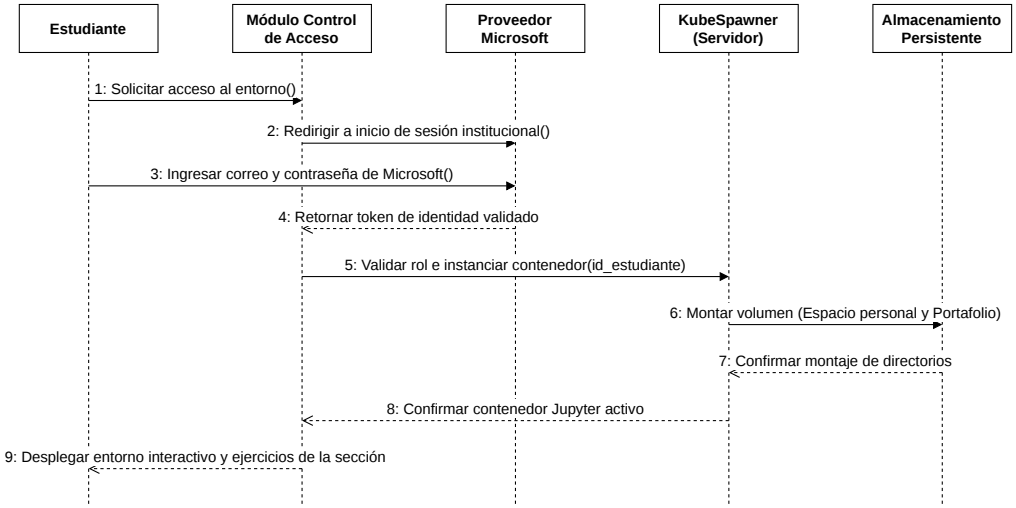


Figura 3: Diagrama de secuencia DS-01: Autenticación institucional y aprovisionamiento del entorno de trabajo.

5.3.2. Entrega de ejercicio al portafolio de evaluación (CU-02)

La Figura 4 representa cómo el estudiante entrega o actualiza un ejercicio desde su directorio de desarrollo al área de revisión (*shipping*). Al recibir la solicitud, el Módulo de Espacio de Trabajo consulta en la base de datos académica el plazo límite de la actividad y, mediante un fragmento de alternativa (`alt`), evalúa su vigencia. Si el envío es oportuno, el archivo se copia del *Espacio personal* al *Portafolio de evaluación* (reemplazando cualquier envío previo de esa sesión) y se registra la marca de tiempo. Vencido el plazo, la operación se rechaza y la última versión almacenada queda consolidada de forma inmutable.

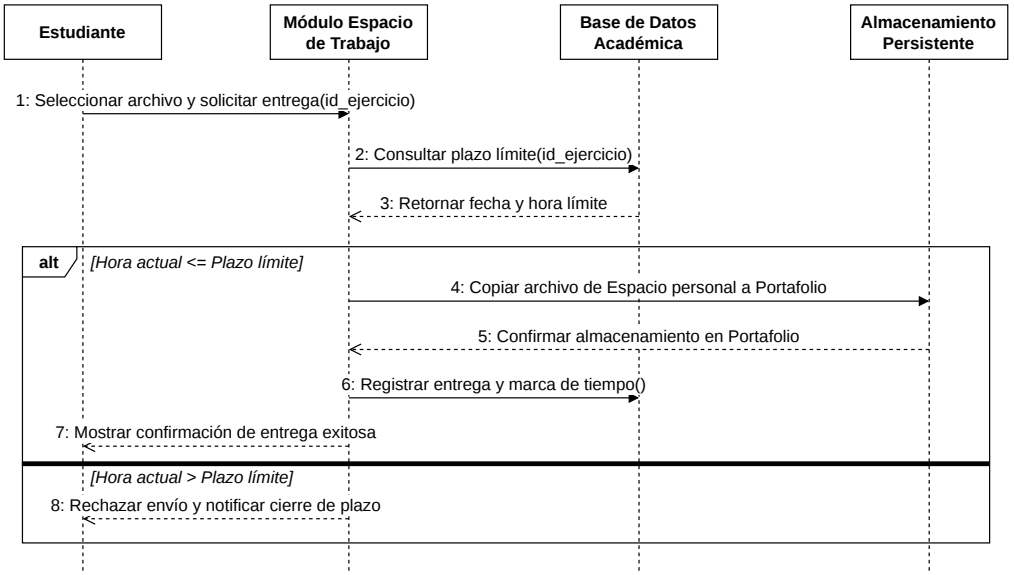


Figura 4: Diagrama de secuencia DS-02: Entrega de ejercicio al portafolio con validación de plazo límite.

5.3.3. Consulta de código en sala y revisión de portafolios (CU-03 y CU-05)

La Figura 5 modela las dos modalidades de supervisión del profesor en el Módulo de Seguimiento Docente. Primero, el docente selecciona una sección y el sistema recupera la nómina de estudiantes con su matriz de entregas. Luego, un fragmento condicional (`alt`) distingue el objetivo. Para asistir una duda en sesión (CU-03), el módulo accede en solo lectura al *Espacio personal* del alumno y muestra la última versión autoguardada de su código en tiempo casi real. Para revisar el avance en el portafolio (CU-05), durante la sesión o tras el cierre del plazo, carga una copia de trabajo desde el *Portafolio de evaluación* con ejecución y edición de prueba, para verificar que la salida coincida con lo solicitado sin modificar el archivo entregado.

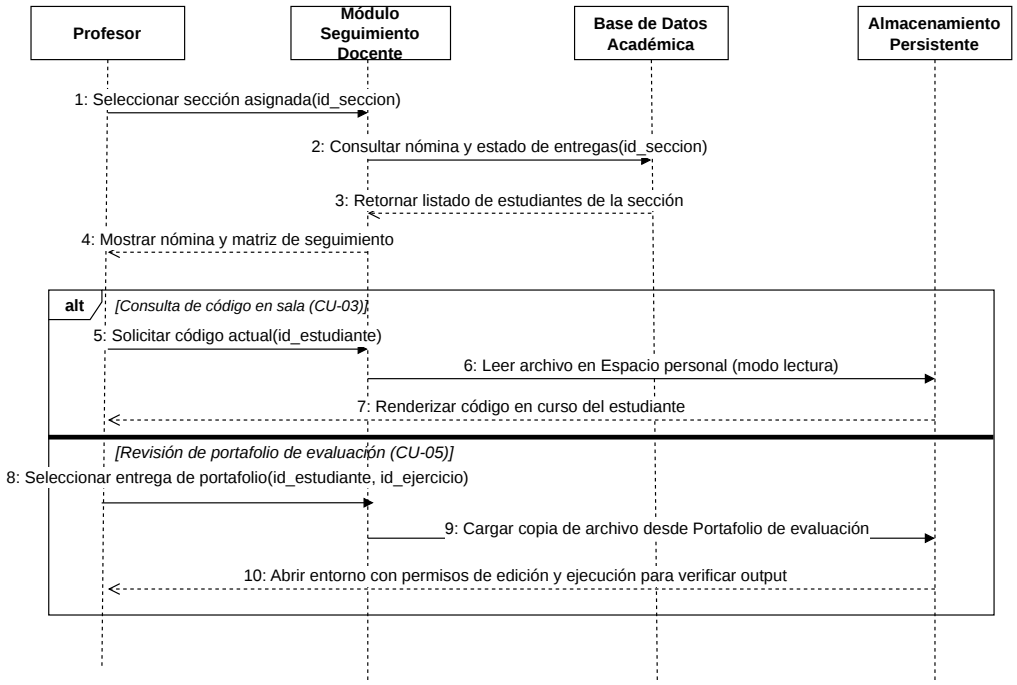


Figura 5: Diagrama de secuencia DS-03: Consulta de código en sala y revisión ejecutable de portafolios.

5.3.4. Generación de ejercicio en sala con plazo límite (CU-04)

La Figura 6 describe la publicación de una actividad práctica para una sección. El profesor ingresa las instrucciones, adjunta el archivo base y define la fecha y hora límite de recepción. El Módulo de Seguimiento Docente registra los metadatos y el plazo en la base de datos académica y luego guarda la plantilla en el sistema de archivos persistente, preparando los directorios del portafolio para recibir los envíos de la sección.

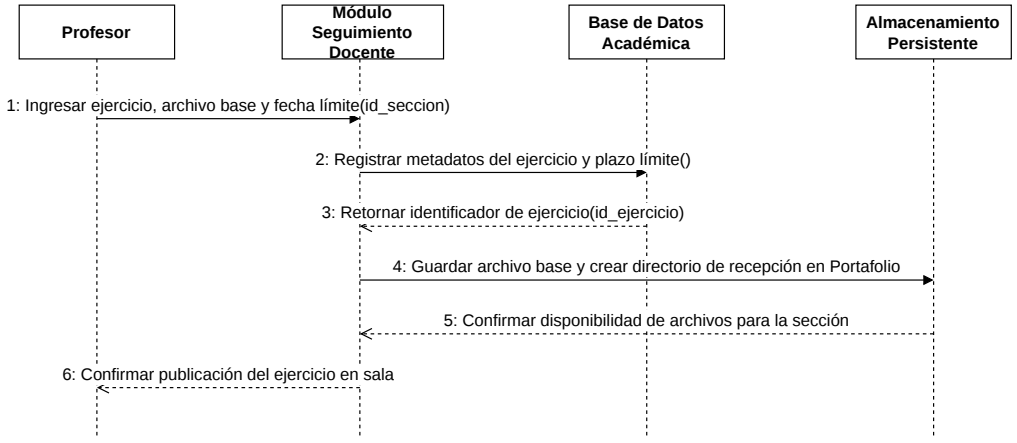


Figura 6: Diagrama de secuencia DS-04: Generación y publicación de ejercicio en sala con plazo límite.

5.4. Diagramas de estado

Se modelan los estados de los dos elementos que gobiernan el consumo de recursos del servidor y la trazabilidad académica: el contenedor efímero del estudiante y la entrega de un ejercicio al portafolio de evaluación.

5.4.1. Ciclo de vida del contenedor del estudiante

La Figura 7 presenta los estados del entorno de programación del alumno con contenedores efímeros y almacenamiento desacoplado. Sin sesión activa, el entorno está en el estado *Sin contenedor activo*: en el servidor solo existen sus directorios persistentes, sin consumir RAM, procesador ni capas de contenedores detenidos.

Cuando el estudiante inicia sesión con sus credenciales institucionales, el sistema pasa a *En aprovisionamiento*: `KubeSpawner` crea un contenedor nuevo desde la imagen base y monta el volumen del alumno (*Espacio personal* y *Portafolio de evaluación*). Montado el almacenamiento, el entorno queda *En ejecución* mientras dure la sesión. Si el estudiante cierra sesión o alcanza el límite de inactividad, pasa a *En destrucción de contenedor*: se desmonta el volumen para resguardar los archivos, se elimina el contenedor y se vuelve al estado inicial hasta un nuevo inicio de sesión.

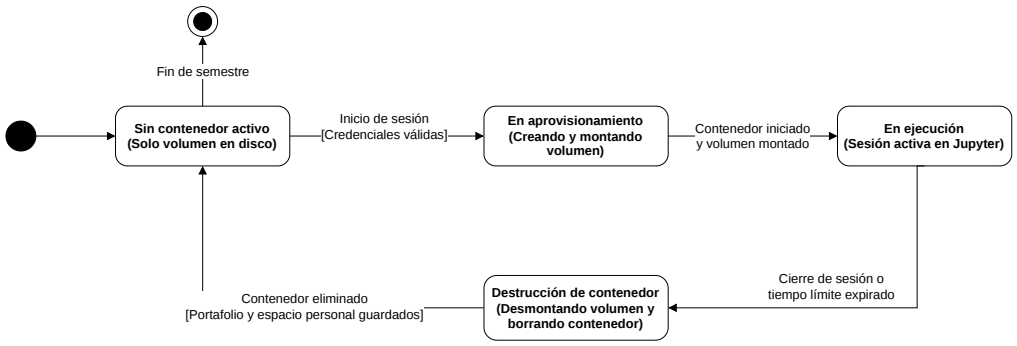


Figura 7: Diagrama de estado DE-01: Ciclo de vida del contenedor efímero del estudiante.

5.4.2. Ciclo de vida del ejercicio y entrega al portafolio

La Figura 8 modela los estados de una actividad práctica desde su publicación hasta su verificación docente. Parte en *Publicado en sala*, cuando el profesor crea el ejercicio y fija su plazo, y pasa a *En desarrollo* en el *Espacio personal* del estudiante al ser descargado.

Si el alumno envía antes del vencimiento, la actividad pasa a *Entregado en portafolio* y se guarda en el área de *shipping*, donde puede reentregar mientras el plazo siga abierto. Al cumplirse el plazo, la última entrega pasa a *Entrega consolidada* y es inmutable para el estudiante. Desde ahí, cuando el profesor abre y ejecuta la entrega sobre una copia de trabajo, alcanza el estado *Revisado y ejecutado por docente* sin alterar el respaldo original. Si el plazo vence sin envío desde *Publicado en sala* o *En desarrollo*, la actividad pasa al estado terminal *No entregado*.

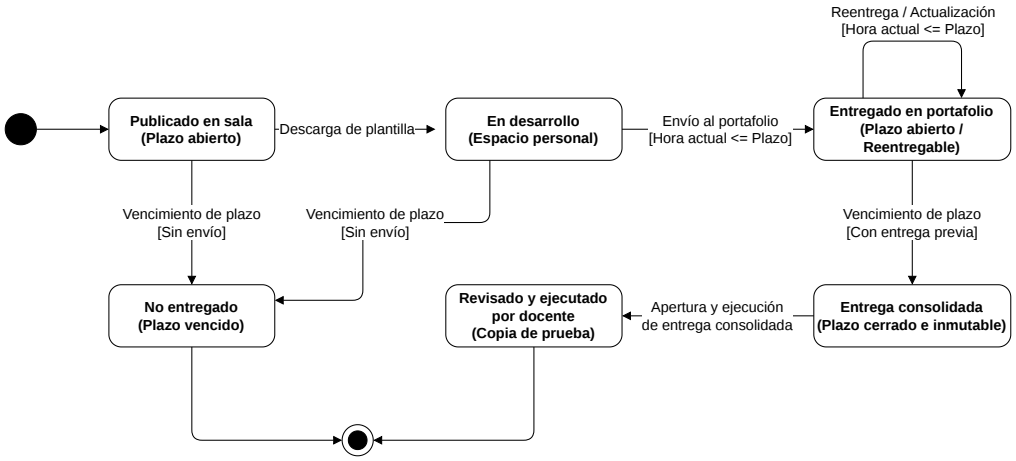


Figura 8: Diagrama de estado DE-02: Ciclo de vida del ejercicio en sala y su entrega al portafolio.

5.5. Modelo conceptual de datos

La información del sistema se organiza en torno a la vinculación entre la carga académica semestral y el almacenamiento persistente dual de cada alumno. La Figura 9 presenta el modelo conceptual como diagrama de clases de dominio UML 2.0, con las entidades, sus atributos principales y las multiplicidades que rigen la trazabilidad por estudiante y sección.

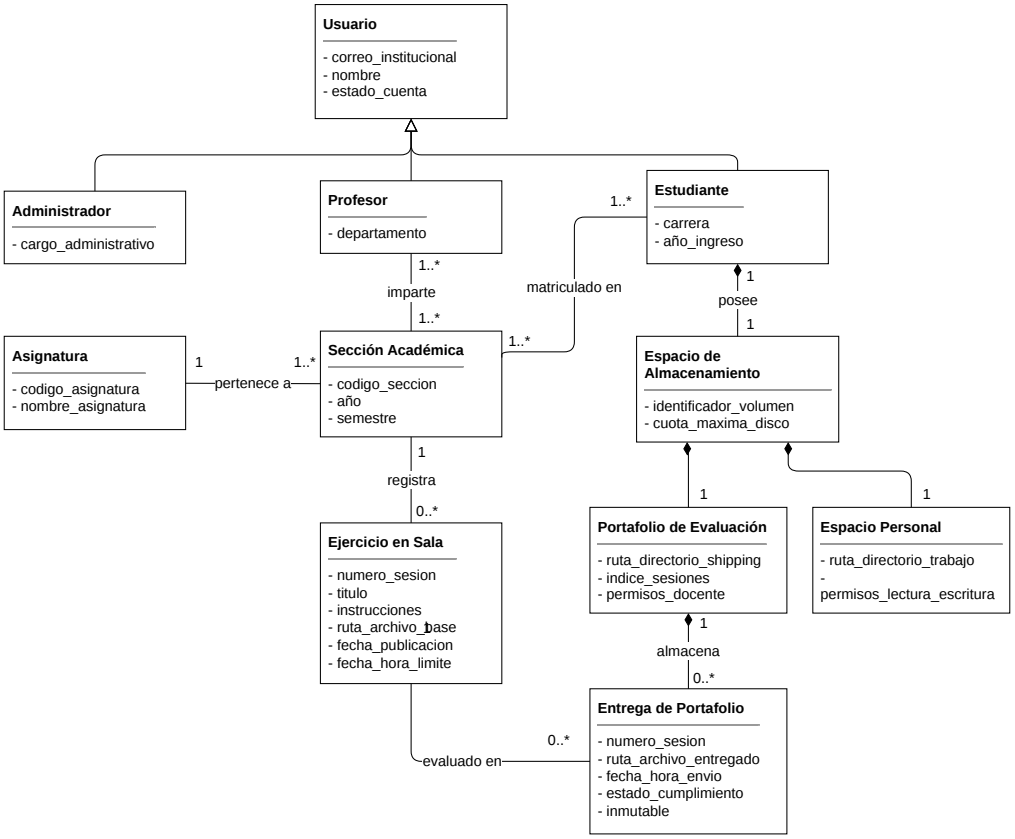


Figura 9: Modelo conceptual de datos del Sistema de Apoyo a la Práctica de Programación.

El modelo se articula en los siguientes grupos de entidades:

- **Jerarquía de usuarios:** *Usuario* centraliza los atributos comunes (correo institucional, nombre y estado de la cuenta) y se especializa en tres subclases: *Administrador*, *Profesor* y *Estudiante*.
- **Estructura académica:** una *Asignatura* del Plan Común (como *Introducción a la programación* o *Programación*) se divide en una o más *Secciones Académicas* por semestre. Cada sección tiene uno o más *Profesores* y agrupa a múltiples *Estudiantes* (relaciones de muchos a muchos), lo que permite aislar los permisos de consulta docente según las secciones de cada profesor.
- **Composición del almacenamiento:** cada *Estudiante* posee de forma exclusiva un *Espacio de Almacenamiento* persistente en el servidor, compuesto por un *Espacio Personal* (lectura y escritura para la práctica diaria) y un *Portafolio de Evaluación* (directorio de *shipping*, que define el índice de sesiones y los permisos de acceso e inspección docente).
- **Trazabilidad de ejercicios y entregas por sesión:** una *Sección Académica* registra cero o más *Ejercicios en Sala*, que identifican la sesión de laboratorio a la que pertenecen, la ruta

del archivo base y la fecha y hora límite. La *Entrega de Portafolio* vincula el *Portafolio de Evaluación* del alumno con el *Ejercicio en Sala* correspondiente y registra el número de sesión, la ruta del archivo entregado, la marca de tiempo, el estado de cumplimiento y un indicador booleano de inmutabilidad.

5.6. Especificación de requisitos no funcionales

Las restricciones técnicas de operación derivan del despliegue en un servidor central con contenedores efímeros, la persistencia desacoplada de los archivos, la seguridad al ejecutar código arbitrario y la preparación para un escalamiento futuro. Se especifican a continuación los ocho requisitos no funcionales (RNF) con sus umbrales de verificación:

- **RNF-01 (Tiempo de aprovisionamiento y liberación de contenedores efímeros):** el sistema debe entregar el contenedor individual del estudiante en menos de 15 segundos desde la validación de sus credenciales (con la imagen base precargada en el nodo), y destruir todo contenedor que supere 30 minutos de inactividad o cierre su sesión, liberando el 100 % de la memoria RAM asignada al Pod.
- **RNF-02 (Persistencia desacoplada del ciclo de vida del contenedor):** los directorios del *Espacio personal* y del *Portafolio de evaluación* deben residir en volúmenes persistentes del servidor, con una cuota de hasta 500 MB por estudiante, garantizando cero pérdida de archivos tras la destrucción del contenedor efímero al finalizar cada sesión.
- **RNF-03 (Integridad e inmutabilidad del respaldo en el portafolio):** los archivos del *Portafolio de evaluación* solo pueden actualizarse mediante reentregas del estudiante mientras el plazo límite de la sesión siga abierto ($t_{\text{envío}} \leq t_{\text{límite}}$), y quedan bloqueados con permisos de solo lectura una vez vencido. Cuando el profesor abra una entrega para ejecutarla y verificar su salida, el sistema debe operar sobre una copia temporal aislada que garantice cero modificaciones sobre el archivo original entregado.
- **RNF-04 (Cuotas de recursos y concurrencia para sesiones de laboratorio):** el sistema debe almacenar en disco los volúmenes de toda la matrícula semestral (cerca de 500 estudiantes) y garantizar en un único nodo, como mínimo, la operación simultánea de una a dos salas de laboratorio (entre 40 y 80 contenedores concurrentes) [32], considerando a los 500 usuarios como cota superior teórica de dimensionamiento.
- **RNF-05 (Orquestación en nodo único preparada para escalamiento horizontal):** el sistema debe desplegarse sobre Kubernetes de un solo nodo con `KubeSpawner` y *PersistentVolumeClaims*, de modo que incorporar nuevos nodos de trabajo al clúster requiera cero modificaciones en el código fuente de los módulos académicos.
- **RNF-06 (Acceso web sin configuración local):** el 100 % de las funcionalidades de edición de código, ejecución interactiva y envío al portafolio deben operar a través de HTTPS (puerto 443) desde un navegador web estándar compatible con HTML5, sin instalaciones locales en el equipo cliente.
- **RNF-07 (Aislamiento docente por sección y sincronización casi real):** el sistema debe restringir las consultas de código en sala y la revisión de portafolios únicamente a las secciones asignadas al profesor autenticado. Además, el entorno del estudiante debe configurar un autoguardado automático de 60 segundos como máximo para respaldar la visualización docente en tiempo casi real.

- **RNF-08 (Aislamiento de seguridad para ejecución de código arbitrario):** como la plataforma ejecuta código escrito por estudiantes y probado por docentes, cada contenedor debe ejecutarse con un usuario sin privilegios de superusuario (`non-root`, p. ej., UID 1000), con escalamiento de privilegios deshabilitado (`allowPrivilegeEscalation: false`) y con políticas de red que bloqueen el acceso directo hacia los contenedores de otros alumnos o servicios internos del servidor.

La Tabla 12 resume los RNF con su umbral verificable y su categoría de calidad.

Tabla 12: Resumen de los requisitos no funcionales del sistema.

Código	Nombre del requisito	Umbral / Métrica	Categoría
RNF-01	Aprovisionamiento y liberación efímera	Inicio $<$ 15 s, inactividad 30 min	Eficiencia
RNF-02	Persistencia desacoplada del contenedor	Cuota 500 MB, 0 % pérdida	Fiabilidad
RNF-03	Integridad del respaldo en portafolio	Inmutable tras $t_{\text{límite}}$, copia docente	Integridad
RNF-04	Concurrencia en sala y cuotas por Pod	40–80 activos (500 máx.)	Rendimiento
RNF-05	Orquestación en nodo único escalable	K8s/`KubeSpawner`, 0 cambios al escalar	Escalabilidad
RNF-06	Acceso web sin configuración local	100 % vía navegador (HTTPS/HTML5)	Usabilidad
RNF-07	Aislamiento por sección y tiempo casi real	Autoguardado $\leq$ 60 s	Privacidad
RNF-08	Ejecución segura de código (*sandboxing*)	Contenedor `non-root`, red aislada	Seguridad

Referencias

[1] Universidad de Valparaíso, Admisión: Oferta de carreras, lista oficial de las 10 carreras que incluyen asignaturas de programación en su currículum: Ingeniería Civil Informática, Ingeniería Civil Biomédica, Ingeniería Civil Industrial, Ingeniería Civil Oceánica, Ingeniería Civil, Ingeniería Civil Matemática, Ingeniería Ambiental, Ingeniería en Estadística, Licenciatura en Física y Licenciatura en Matemáticas. Fecha de acceso: 01 de septiembre de 2026 (2026).
URL `https://admision.uv.cl/carreras`

[2] A. Luxton-Reilly, Simon, I. Albluwi, B. A. Becker, M. Giannakos, A. N. Kumar, L. Ott, J. Paterson, M. J. Scott, J. Sheard, C. Szabo, Introductory programming: A systematic literature review, in: Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE 2018 Companion), ACM, New York, NY, USA, 2018, pp. 55–106. `doi:10.1145/3293881.3295779`.

[3] R. P. Medeiros, G. L. Ramalho, T. P. Falcão, A systematic literature review on teaching and learning introductory programming in higher education, IEEE Transactions on Education 62 (2) (2019) 77–90. `doi:10.1109/TE.2018.2864133`.

[4] P. J. Guo, Codeopticon: Real-time, one-to-many human tutoring for computer programming, in: Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology (UIST '15), ACM, 2015, pp. 599–608. doi:10.1145/2807442.2807469.

[5] J. Flemming, Jupyterhub and autograding on bare-metal lab servers, Technical report, Zwickau University of Applied Sciences (2022). doi:10.34806/6f2w-v119.

[6] M. Linhuber, J. P. Bernius, S. Krusche, On-premise kubernetes in software engineering education: An experience report, in: Proceedings of the 48th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET '26), 2026, p. 11. doi:10.1145/3786580.3786953.

[7] J. W. Johnson, Benefits and pitfalls of jupyter notebooks in the classroom, in: The 21st Annual Conference on Information Technology Education (SIGITE '20), Association for Computing Machinery, New York, NY, USA, 2020, pp. 32–37. doi:10.1145/3368308.3415397.

[8] Z. Xu, Teaching heterogeneous and parallel computing with google colab and raspberry pi clusters, in: Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023), Association for Computing Machinery, New York, NY, USA, 2023, pp. 308–313. doi:10.1145/3624062.3624095.

[9] Project Jupyter, KubeSpawner: JupyterHub Spawner for Kubernetes, accedido: 14-octubre-2026 (2026).
URL https://kubespawner.readthedocs.io/en/latest/

[10] The Kubernetes Authors, Persistent Volumes, accedido: 14-octubre-2026 (2024).
URL https://kubernetes.io/docs/concepts/storage/persistent-volumes/

[11] Project Jupyter, What is the Jupyter Notebook?, accedido: 13-octubre-2026 (2024).
URL https://jupyter-notebook.readthedocs.io/en/latest/notebook.html

[12] Project Jupyter, JupyterLab Documentation: Overview, accedido: 13-octubre-2026 (2024).
URL https://jupyterlab.readthedocs.io/en/stable/getting_started/overview.html

[13] Project Jupyter, Jupyter Server Architecture and Public Server, accedido: 13-octubre-2026 (2024).
URL https://jupyter-server.readthedocs.io/en/latest/operators/public-server.html

[14] Project Jupyter, JupyterHub Technical Overview, accedido: 13-octubre-2026 (2026).
URL https://jupyterhub.readthedocs.io/en/stable/reference/technical-overview.html

[15] C. Pahl, A. Brogi, J. Soldani, P. Jamshidi, Cloud container technologies: A state-of-the-art review, IEEE Transactions on Cloud Computing 7 (3) (2019) 677–692. doi:10.1109/TCC.2017.2702586.

[16] Docker Inc., Docker Overview: Architecture and Concepts, accedido: 13-octubre-2026 (2026).
URL https://docs.docker.com/get-started/overview/

[17] M. van Steen, A. S. Tanenbaum, Distributed Systems, 4th Edition, Maarten van Steen, 2023.
URL https://www.distributed-systems.net/index.php/books/ds4/

[18] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, O'Reilly Media, Sebastopol, CA, 2017.

[19] Docker Inc., Resource constraints: Limit a container's resources, accedido: 14-octubre-2026 (2024).
URL https://docs.docker.com/engine/containers/resource_constraints/

[20] Docker Inc., Compose Deploy Specification, accedido: 14-octubre-2026 (2026).
URL https://docs.docker.com/reference/compose-file/deploy/

[21] Docker Inc., Swarm mode key concepts, accedido: 14-octubre-2026 (2024).
URL https://docs.docker.com/engine/swarm/key-concepts/

[22] The Kubernetes Authors, Kubernetes Components, accedido: 14-octubre-2026 (2024).
URL https://kubernetes.io/docs/concepts/overview/components/

[23] The Kubernetes Authors, Workload Autoscaling in Kubernetes, accedido: 14-octubre-2026 (2024).
URL https://kubernetes.io/docs/concepts/workloads/autoscaling/

[24] Docker Inc., Manage data in Docker: Storage Overview, accedido: 14-octubre-2026 (2024).
URL https://docs.docker.com/engine/storage/

[25] The Kubernetes Authors, Volumes in Kubernetes, accedido: 14-octubre-2026 (2024).
URL https://kubernetes.io/docs/concepts/storage/volumes/

[26] Project Jupyter, Zero to JupyterHub with Kubernetes: Customizing User Storage, accedido: 14-octubre-2026 (2026).
URL https://z2jh.jupyter.org/en/stable/jupyterhub/customizing/user-storage.html

[27] J. C. Paiva, J. P. Leal, Á. Figueira, Automated assessment in computer science education: A state-of-the-art review, ACM Transactions on Computing Education 22 (3) (2022) 34:1–34:40. doi:10.1145/3513140.

[28] Y.-C. Tu, V. Terragni, E. Tempero, A. Shakil, A. Meads, N. Giacaman, A. Fowler, K. Blincoe, Github in the classroom: Lessons learnt, in: Proceedings of the 24th Australasian Computing Education Conference (ACE '22), ACM, 2022, pp. 163–172. doi:10.1145/3511861.3511879.

[29] H. A. Olstad, B. R. Krogstie, X. Su, R. Hjelsvold, Enabling computing students to recognize acquired competencies with eportfolios, Computer Science Education 35 (3) (2025) 1–28. doi:10.1080/08993408.2024.2353482.

[30] M. B. Garcia, Self-coded digital portfolios as an authentic project-based learning assessment in computing education: Evidence from a web design and development course, Education Sciences 15 (9) (2025) 1150. doi:10.3390/educsci15091150.

[31] R. Zhi, T. W. Price, S. Marwan, A. Milliken, T. Barnes, M. Chi, Exploring the impact of worked examples in a novice programming environment, in: Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19), ACM, New York, NY, USA, 2019, pp. 98–104. doi:10.1145/3287324.3287385.

[32] Escuela de Ingeniería Informática, Universidad de Valparaíso, Infraestructura y laboratorios, fecha de acceso: 01 de septiembre de 2026 (2026).
URL https://informatica.uv.cl/escuela/infraestructura